CENTRO ESTADUAL DE EDUCAÇÃO TECNOLÓGICA PAULA SOUZA
FACULDADE DE TECNOLOGIA DA BAIXADA SANTISTA "RUBENS LARA"
CURSO SUPERIOR DE TECNOLOGIA EM CIÊNCIA DE DADOS
PROGRAMA DE INICIAÇÃO CIENTÍFICA

PETERSON CARARA JUNIOR

FORMALIZAÇÃO DE MÁQUINAS DE ESTADOS FINITOS COMPONÍVEIS EM HASKELL: SEGURANÇA DE TIPOS E COMPLETUDE FUNCIONAL COM TIPOS ALGÉBRICOS, STATE MONAD E A BIBLIOTECA CREM

Santos - SP
2025

PETERSON CARARA JUNIOR

FORMALIZAÇÃO DE MÁQUINAS DE ESTADOS FINITOS COMPONÍVEIS EM HASKELL: SEGURANÇA DE TIPOS E COMPLETUDE FUNCIONAL COM TIPOS ALGÉBRICOS, STATE MONAD E A BIBLIOTECA CREM

Projeto de pesquisa apresentado à Faculdade de Tecnologia da Baixada Santista "Rubens Lara", do Curso Superior de Tecnologia em Ciência de Dados, no Programa de Iniciação Científica.

Orientador: Prof. Felipe Canarozzo Lourenço

Santos - SP

2025

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
ADT	Algebraic Data Type (Tipo de Dados Algébrico)
CREM	Crème de la Crem: Composable Representable Executable Machines
DDD	Domain-Driven Design
DFSM	Deterministic Finite State Machine
FATEC BS	Faculdade de Tecnologia da Baixada Santista Rubens Lara
FSM	Finite State Machine (Máquina de Estados Finitos)
I/O	Input/Output
PFP	Programação Funcional Pura

RESUMO

Este relatório parcial descreve a formalização, em Haskell, de uma máquina de estados finitos de aviônica de um veículo aéreo não tripulado (UAV), em quatro camadas: tipos algébricos com casamento de padrões, State Monad com composição de Kleisli, a biblioteca CREM e a composição de máquinas. A aviônica é um domínio de exemplo escolhido pelo projeto, sem requisitos reais de segurança de voo. Os resultados são os seguintes. A completude da função de transição é verificada pelo compilador como aviso, que o projeto promove a erro. Uma transição fora da topologia do CREM é rejeitada em compilação, o que foi testado nos 16 pares de vértices (8 compilam e 8 são rejeitados) contra uma especificação escrita a partir dos requisitos. As leis de Functor, Applicative e mônada foram demonstradas para a State Monad do projeto. A composição foi realizada com quatro construtores do CREM (`Basic`, `Sequential`, `Parallel` e `Alternative`) e com `Feedback`, e simulada em malha aberta e fechada, com enumeração de até 168 421 cenários. Um modelo de referência independente do CREM, escrito em Haskell comum, coincidiu com o simulador em 87 381 cenários, e comparou três políticas para o ambiente. Uma comparação com a mesma máquina em C e em C++ indica que, em domínio finito, um teste exaustivo detecta os mesmos defeitos que o sistema de tipos; a vantagem observada do CREM é antecipar a detecção, sem exigir código de teste. A garantia dos tipos é por aresta, e a enumeração revelou uma lacuna do modelo do ambiente: ele reage à entrada da bateria em nível crítico, e não ao fato de a bateria estar em nível crítico.

Palavras-chave: máquinas de estados finitos; Haskell; tipos algébricos; mônadas; CREM.

Sumário

LISTA DE ABREVIATURAS E SIGLAS 1

RESUMO 2

1 INTRODUÇÃO 5
1.1 PROBLEMA DE PESQUISA 6
1.2 JUSTIFICATIVA DO TEMA 6
1.2.1 Hipóteses 6
1.2.2 Objetivo Geral 7
1.2.3 Objetivos Específicos 7
1.2.4 Contribuições, escopo e situação dos objetivos 8

2 REFERENCIAL TEÓRICO INICIAL 8
2.1 Máquinas de Estados Finitos e a Questão da Mutabilidade 9
2.2 Haskell e a Garantia de Corretude em Tempo de Compilação 9
2.2.1 Tipos de Dados Algébricos (ADTs) 10
2.2.2 Pattern Matching e Completude Funcional 10
2.3 Teoria das Categorias: Mônadas, Estado Puro e a Categoria Kleisli 10
2.3.1 Mônadas: Origem Categorial, Leis e a Abstração do Efeito de Estado . 10
2.3.2 A Categoria Kleisli para a Composição de Transições 11
2.4 Composabilidade Arquitetural e o Framework CREM 11

3 PROCEDIMENTOS METODOLÓGICOS 12

4 ESTADO DA ARTE 14
4.1 O problema nas fontes 14
4.2 Quatro abordagens 14
4.3 Quadro comparativo 15
4.4 Lacunas 15

5 PROGRESSO PARCIAL 16
5.1 Correspondência entre Tipos de Dados Algébricos e os Conjuntos Q e Sigma . 16
5.2 A Função de Transição delta como Morfismo em Hask 16
5.3 Sequenciação Inicial e Validação Empírica 17
5.4 Da Composição Ingênua à Necessidade de uma Abstração Monádica 17
5.5 Reformulação Teórica da FSM sobre a State Monad 18
5.6 Sequenciação via Notação do e a Função mapM 18
5.7 Fundamentação Categorial: a Mônada como Monoide na Categoria dos Endofuntores 19

5.8 Estudo de Caso: Topologia Promovida ao Nível de Tipos 20
5.9 Verificação Formal de Segurança de Tipos em Tempo de Compilação 21
5.10 Execução, Validação Cruzada e Composição 23
5.11 Simulador em Malha Aberta e Fechada e Robustez dos Testes 24
5.12 Conformidade com a Especificação do Domínio e Matriz de Rejeição 26
5.13 Cadeia da Bateria e Descarte do Registrador de um Slot 28
5.14 Comparação com a Mesma Máquina em C 29
5.15 Limitações Identificadas e Encaminhamentos 30
5.16 Síntese Geral e Validação Preliminar das Hipóteses 31

6 CRONOGRAMA 33

DECLARAÇÕES 33

7 REFERÊNCIAS 35

APÊNDICE A — REPRODUÇÃO DOS RESULTADOS 36

1 INTRODUÇÃO

As Máquinas de Estados Finitos (FSMs) constituem uma das abstrações mais duradouras e essenciais da Ciência da Computação. Elas fornecem o formalismo matemático rigoroso para a modelagem de sistemas cujo comportamento é estritamente determinado por um conjunto discreto de estados e regras de transição. O modelo é crucial para a especificação do comportamento de sistemas reativos, abrangendo desde protocolos de rede até a lógica de domínio em arquiteturas de software complexas. Conforme Perone e Karachalias (2023), é possível construir arquiteturas de software a partir da composição de máquinas de estados.

No entanto, a tradução desse modelo abstrato para linguagens de programação de propósito geral do paradigma imperativo, como C++, é frequentemente marcada pela dependência de variáveis mutáveis, tipicamente campos de classe ou variáveis globais, para armazenar o estado atual. Essa prática compromete a pureza do modelo matemático. Segundo Malakhovski e Shalyto (2009), a implementação em linguagens imperativas não permite a aplicação direta dos métodos tradicionais de FSMs em um ambiente funcional. A dependência da mutabilidade e de tipos fracos para o estado expõe o sistema a erros lógicos e transições inválidas, delegando a responsabilidade de verificar a corretude à fase de testes em tempo de execução.

O presente projeto de pesquisa propõe resolver essa discrepância fundamental através da formalização e implementação de FSMs em Haskell. Por ser uma linguagem puramente funcional (Hutton, 2016), Haskell exige que a função de transição seja implementada como uma função pura. O diferencial reside no seu sistema de tipos avançado: a utilização de Tipos de Dados Algébricos (ADTs) é essencial para a modelagem de dados (Milewski, 2020), permitindo que os conjuntos finitos de estados e eventos sejam mapeados diretamente para tipos de dados. Essa correspondência de tipo não é apenas uma convenção, mas uma imposição que, por meio da verificação de exaustividade do *pattern matching*, viabiliza a verificação da correção das funções de transição no estágio de compilação (Malakhovski & Shalyto, 2009).

Além da segurança de tipos básica, a abordagem em Haskell permite a exploração de conceitos avançados para o tratamento do estado. A capacidade de sequenciar transições e lidar com o estado de forma pura é resolvida através de mônadas. Essa abstração, que expressa de forma concisa programas que manipulam estado (Wadler, 1992), permite o tratamento puro do estado através de estruturas como a `State Monad` (Milewski, 2020). Esse arcabouço monádico, estruturado pela Categoria Kleisli (Maia, 2022), fornece o formalismo necessário para a composição segura de transições.

A capacidade de composabilidade é o ponto culminante desta abordagem. O *framework* Crem (Crème de la Crem: Composable Representable Executable Machines), desenvolvido por Perone e Karachalias (2023), ilustra como os *advanced type-level features* de Haskell podem ser usados para especificar transições de estado permitidas e proibidas. Essa especificação transforma as regras de domínio em propriedades intrínsecas ao tipo de dados, permitindo a composição de máquinas de estados independentes (Perone e Karachalias, 2023), resultando

em um sistema robusto e modular, onde as transições fora da topologia declarada são rejeitadas pelo compilador antes da execução.

1.1 PROBLEMA DE PESQUISA

Considerando o descompasso entre a formalidade matemática das FSMs e a fragilidade de tipos nas implementações tradicionais (Malakhovski & Shalyto, 2009), o problema de pesquisa é formulado como:

> De que forma a linguagem de programação funcional pura Haskell, explorando seu sistema de tipos avançados (Tipos de Dados Algébricos) e suas abstrações de ordem superior (mônadas e Categoria Kleisli), pode ser utilizada para obter uma formalização de Máquinas de Estados Finitos que ofereça garantias verificáveis de correção (*type safety*) e completude (*completeness*) na especificação das funções de transição, e que suporte a composição de FSMs complexas de maneira modular e representável?

1.2 JUSTIFICATIVA DO TEMA

A relevância do projeto reside em transferir a verificação da corretude da FSM da fase de testes para a fase de compilação. Segundo Malakhovski e Shalyto (2009), uma das vantagens da implementação funcional é a verificação da correção do código das funções de transição já no estágio de compilação, o que é possível pelo uso de Tipos de Dados Algébricos (ADTs) para representar eventos e estados.

Além disso, a estrutura de Haskell é baseada na Teoria das Categorias (Milewski, 2020). Essa base formal permite lidar com o estado de forma pura através de mônadas, que foram originalmente desenvolvidas por teóricos de categorias para expressar concisamente certos aspectos da álgebra universal (Wadler, 1992). Formalmente, uma mônada é definida como um Monóide na Categoria dos Endofuntores (Jahn, 2020) e permite a mudança de estado de forma pura (Milewski, 2020). Essas estruturas possibilitam a composição segura de transições via a Categoria Kleisli (Maia, 2022; Jahn, 2020). Essa composabilidade é crucial, pois permite a construção de arquiteturas de software a partir da composição de máquinas de estados independentes (Perone e Karachalias, 2023), o que contribui para a modularidade e a coerência do domínio do *software*.

1.2.1 Hipóteses

Hipótese de completude funcional: o uso de tipos de dados algébricos e de casamento de padrões exaustivo permite verificar, em compilação, que a função de transição cobre todas as combinações de (Estado $\times$ Evento); o compilador emite um aviso, que o projeto promove a erro. Hipótese de rejeição antecipada: com a topologia promovida ao nível dos tipos (CREM), uma

transição fora da topologia é rejeitada em compilação. A vantagem esperada, em comparação com a abordagem imperativa, é antecipar a detecção do defeito e dispensar código de teste; ela será avaliada comparando o que um compilador e um teste exaustivo detectam (seção 5.14). Hipótese de composabilidade e modulação: a composição de máquinas com os construtores do CREM, apoiada em abstrações como as mônadas, permite criar máquinas de estados finitos componíveis de forma puramente funcional, em que cada máquina conserva a garantia de suas transições (Perone e Karachalias, 2023).

1.2.2 Objetivo Geral

O objetivo geral desta Iniciação Científica é formalizar e implementar Máquinas de Estados Finitos Determinísticas (DFSMs) em Haskell, explorando o sistema de tipagem avançado e as estruturas funcionais da linguagem para garantir a segurança e a completude na definição das funções de transição, com ênfase na demonstração de um modelo que suporte a composição de sistemas.

1.2.3 Objetivos Específicos

Para alcançar o objetivo geral de formalizar e implementar FSMs de forma segura e componível em Haskell, o projeto se desdobra nos seguintes objetivos específicos. O texto conserva a redação da proposta aprovada, no tempo verbal da época; ao fim de cada objetivo, a frase "Situação" registra o que foi feito até agora.

O primeiro objetivo visa o alicerce do projeto: definir modelos de dados em Haskell que traduzam com precisão os elementos formais de uma FSM. Esta formalização será realizada através da criação de Tipos de Dados Algébricos (ADTs), nos quais os conjuntos finitos de Estados (Q) e Eventos (Σ) serão mapeados diretamente para Tipos Soma. Essa técnica visa incorporar o rigor do modelo matemático ao código, seguindo o princípio de que ADTs são o conceito mais importante na modelagem funcional (Milewski, 2020). Situação: cumprido (seção 5.1).

O segundo objetivo trata da validação da lógica interna da máquina: implementar o mecanismo de processamento de sequências de entrada e a função de transição (δ). Serão exploradas as funções de alta ordem e, sobretudo, o *pattern matching* exaustivo para definir a função δ : Estado $\times$ Evento $\rightarrow$ Estado. Essa abordagem é fundamental para atingir a Hipótese de Completude Funcional, pois o compilador de Haskell verifica a exaustividade (como aviso, que o projeto promove a erro de compilação), de modo que todo par $(\text{Estado}, \text{Evento})$ seja tratado, transferindo a verificação da correção do código das funções de transição para o estágio de compilação (Malakhovski & Shalyto, 2009). Situação: cumprido, com a ressalva de que o compilador emite aviso, e o projeto o promove a erro (seções 5.2 e 5.12).

Em seguida, o terceiro objetivo foca na gestão do estado e na composabilidade: explorar a aplicação do *State Monad* e da Categoria Kleisli. Será demonstrado como a `State Monad`

permite gerenciar a sequência de mudanças de estado de forma pura, encapsulando o estado e garantindo a mudança de estado de forma pura (Milewski, 2020). Esta etapa é crítica para lidar com o histórico de transições. Além disso, a Categoria Kleisli será utilizada como base teórica para compor transições monádicas, fornecendo o formalismo para encadear logicamente as transições de estado, como sugerido por Milewski (2020) e Maia (2022). Situação: cumprido para a State Monad; as leis de mônada, Functor e Applicative foram demonstradas (seção 5.7).

Finalmente, o quarto objetivo é a validação prática da abordagem: demonstrar a composabilidade de máquinas de estados por meio de um estudo de caso. Este estudo de caso ilustrará como a tipagem de Haskell pode ser usada para garantir transições válidas e o encapsulamento do domínio, permitindo que a lógica do *software* seja estruturada como uma composição de máquinas de estados modulares (Perone e Karachalias, 2023). O resultado esperado é um sistema onde as regras de transição — inclusive a especificação de transições de estado permitidas e proibidas (Perone e Karachalias, 2023) — são validadas pelo compilador, confirmando o potencial das estruturas funcionais para o desenvolvimento de *software* robusto. Situação: parcial; cinco construtores do CREM foram exercitados, e o construtor `Kleisli` não foi (seções 5.10, 5.11 e 5.13).

1.2.4 Contribuições, escopo e situação dos objetivos

Esta subseção resume o que o projeto acrescenta ao que já existe nas fontes, os limites do estudo de caso e a situação de cada objetivo. As seções 1 a 3 mantêm a redação da proposta; a situação atual está na Tabela 1.

Contribuições deste projeto, em relação a Perone e Karachalias (2023), que apresentam o CREM e um exemplo de carrinho de compras: (i) um estudo de caso de aviônica com teste de rejeição para cada um dos 16 pares de vértices, confrontado com uma especificação escrita a partir dos requisitos; (ii) a demonstração das leis de Functor, Applicative e mônada para a State Monad usada com o CREM; (iii) um simulador puro, em malha aberta e fechada, com o construtor `Feedback` e validação por enumeração limitada; (iv) a delimitação do alcance da garantia dos tipos, que é por aresta e não cobre a associação entre evento e destino. Não foi feita a busca de literatura que permitiria afirmar ineditismo; ver a seção 4.

Escopo. A máquina de aviônica é um exemplo escolhido pelo projeto, a partir de um enunciado próprio, com quatro estados e quatro eventos. Não há requisitos reais de segurança de voo, e os resultados não devem ser lidos como relativos a sistemas de aviônica reais.

2 REFERENCIAL TEÓRICO INICIAL

O presente referencial teórico estabelece a base de conhecimento necessária para a formalização das Máquinas de Estados Finitos (FSMs) no ambiente funcional puro. A abordagem é construída sobre a fusão de quatro domínios conceituais: a Teoria de Autômatos, a Programação Funcional Pura (PFP), a Teoria das Categorias e as Arquiteturas de *Software* Componíveis.

Objetivo específico	Situação	Evidência
1. Modelos de dados (tipos algébricos para Q e Σ)	cumprido	seção 5.1 e módulo `Avionics.Naive`
2. Função de transição δ por casamento de padrões exaustivo	cumprido, com ressalva	16 pares escritos um a um, sem curinga; a exaustividade é aviso do GHC, promovido a erro pelo projeto (seções 5.2 e 5.12)
3. State Monad e categoria de Kleisli	cumprido para a State Monad	leis de mônada, Functor e Applicative demonstradas e testadas (seção 5.7); `>=>` usado em `composeAll`
4. Composabilidade por estudo de caso	parcial	`Basic`, `Sequential`, `Parallel`, `Alternative` e `Feedback` exercitados (seções 5.10 e 5.11); o construtor `Kleisli` do CREM não foi
Fase 4: comparação com a abordagem imperativa	parcial	uma máquina em C e em C++, sete variantes cada (seção 5.14); sem estudo de esforço de manutenção

Tabela 1: Situação dos objetivos.

2.1 Máquinas de Estados Finitos e a Questão da Mutabilidade

As FSMs são o modelo canônico para descrever sistemas sequenciais e reativos, cuja dinâmica é regida por uma quíntupla formal $(Q, \Sigma, \delta, q_0, F)$, onde Q é o conjunto finito de estados, Σ é o alfabeto de entrada (eventos), δ é a rigorosa função de transição, q_0 é o estado inicial, e F é o conjunto de estados finais. A função $\delta : Q \times \Sigma \to Q$ é o coração do sistema, definindo como o autômato se move entre os estados.

A implementação tradicional desse modelo em linguagens de programação imperativas, como C++ ou Java, cria uma divergência perigosa entre o modelo e o código. A representação do estado recorre, invariavelmente, a variáveis mutáveis — tipicamente um campo de classe (se a programação orientada a objetos for usada) ou uma variável global (Malakhovski & Shalyto, 2009). Além disso, os identificadores de estado são frequentemente representados por abstrações de baixo nível, como *strings* ou inteiros, que carecem de garantias de tipo. Essa dependência de estado mutável e a fragilidade de tipos resultam numa tradução que não consegue aplicar diretamente os métodos utilizados na implementação de máquinas de estados finitos em linguagens de programação imperativas no rigoroso domínio funcional (Malakhovski & Shalyto, 2009), expondo a FSM a estados logicamente impossíveis ou transições inválidas que só são detectadas em tempo de execução, através de pesados testes unitários.

2.2 Haskell e a Garantia de Corretude em Tempo de Compilação

O projeto propõe a Programação Funcional Pura (PFP) como a solução para este descompasso. Haskell, uma linguagem puramente funcional, baseia-se na imutabilidade e na ausência de efeitos colaterais. Sua filosofia exige que a função de transição (δ) seja implementada como uma função pura, uma transformação que recebe o estado e o evento e retorna, deterministicamente, o novo estado, preservando a imutabilidade do sistema e produzindo *software* claro, conciso e correto (Hutton, 2016).

2.2.1 Tipos de Dados Algébricos (ADTs)

A segurança formal em Haskell é fornecida por seu sistema de tipos avançado, ancorado nos Tipos de Dados Algébricos (ADTs). Os ADTs permitem que os conjuntos finitos Q e Σ da FSM sejam mapeados diretamente para Tipos Soma, que são considerados o conceito mais importante para a modelagem em Haskell (Milewski, 2020). Por exemplo, se uma FSM possui estados *Ligado* e *Desligado*, o Tipo Soma data Estado = Ligado | Desligado traduz essa finitude para o sistema de tipos. Essa codificação evita o uso de strings e inteiros como identificadores de estado (Malakhovski & Shalyto, 2009), garantindo que o compilador imponha a integridade estrutural da máquina.

2.2.2 Pattern Matching e Completude Funcional

A função de transição é escrita por casamento de padrões (*pattern matching*): cada equação diz qual é o resultado para uma forma possível dos argumentos (Hutton, 2016). Como estado e evento são tipos soma com número finito de construtores, o compilador consegue listar todos os pares (Estado $\times$ Evento) e conferir se cada um tem uma equação. O GHC faz essa conferência, mas o resultado é um aviso, não um erro: quando falta um par, emite o aviso `-Wincomplete-patterns`, que já vem ligado em `-Wall` e informa qual par ficou sem tratamento. O aviso só impede a compilação se o projeto pedir, com `-Werror` ou, de forma mais precisa, `-Werror=incomplete-patterns`. Com GHC 9.6.7, sem a opção o programa compila e o aviso aparece; com a opção, a compilação falha no mesmo ponto.

A completude funcional depende, portanto, de duas condições: a opção de erro deve estar ligada, e a função não pode usar uma equação curinga (_), que cobre qualquer par restante e esconde as omissões. Malakhovski e Shalyto (2009, p. 21) afirmam que, com tipos algébricos, o compilador verifica a completude e a ausência de contradições na função de transição; no exemplo deles, uma equação supérflua faz o compilador emitir um aviso.

2.3 Teoria das Categorias: Mônadas, Estado Puro e a Categoria Kleisli

Em um ambiente funcional puro, onde as transições de estado devem ser implementadas como funções puras, o desafio inerente é a gestão da sequência de transições ao longo do tempo. O projeto aborda este desafio recorrendo à Teoria das Categorias, que é a disciplina matemática que fornece os fundamentos lógicos para a linguagem Haskell. O princípio orientador é que a composição é a essência da programação (Milewski, 2020), e a Teoria das Categorias define o arcabouço para a composição segura e coerente.

2.3.1 Mônadas: Origem Categorial, Leis e a Abstração do Efeito de Estado

As mônadas são o dispositivo arquitetural que permite a Programação Funcional Pura gerenciar efeitos colaterais, como o estado, exceções ou I/O, sem violar a imutabilidade. Segundo

Wadler (1992), as mônadas permitem combinar a flexibilidade de recursos impuros com as garantias da pureza, ao encapsular o efeito.

A sua origem é estritamente matemática: mônadas foram inventadas na década de 1960 por teóricos de categorias para expressar concisamente certos aspectos da álgebra universal (Wadler, 1992). A definição categorial formal e mais abstrata estabelece uma Monad como um Monóide na Categoria dos Endofuntores (Jahn, 2020). Toda mônada deve obedecer a três leis, identidade à esquerda, identidade à direita e associatividade (Hutton, 2016, cap. 12). Bocchino (2025, seções 4.4 a 4.6) mostra que elas equivalem a exigir que a composição de Kleisli seja associativa e tenha identidade. O compilador não verifica essas leis; elas precisam ser demonstradas para cada instância, o que é feito para a State Monad na seção 5.7.

Para a FSM, a `State Monad` é a mônada específica empregada. Ela resolve o dilema do estado ao modelar a transição como uma função de transformação de estado, na qual a função de transição pura δ é encapsulada. Isto permite que a mudança de estado de forma pura seja realizada (Milewski, 2020), pois a função não *modifica* o estado, mas sim *calcula* o novo estado a partir do antigo, retornando o par (Valor, Novo Estado) e mantendo a imutabilidade fora da abstração monádica.

2.3.2 A Categoria Kleisli para a Composição de Transições

Embora as mônadas resolvam o problema do estado encapsulado, a composição dessas funções monádicas (as transições de estado) não pode ser realizada pela composição de funções padrão. É aqui que entra a Categoria Kleisli.

A Kleisli é uma construção categorial específica que é projetada para compor morfismos monádicos, ou seja, funções "sujas"que carregam o efeito do *Monad*. Em vez de funções $A \to B$, a Kleisli compõe morfismos do tipo $A \to MB$ (Maia, 2022), onde M é o Functor Monádico (a `State Monad` no nosso caso) que carrega o estado. Essa categoria fornece as regras e, mais importante, o operador de composição monádica (>=>) que automatiza a logística da sequenciação.

O uso da Categoria Kleisli é crucial para o projeto, pois ele fornece o formalismo para encadear múltiplas transições de estado da FSM de forma segura e elegante. Com a composição de Kleisli, o estado do autômato passa de uma etapa à seguinte sem que o código o manipule explicitamente; no projeto isso aparece em >=>, `composeAll` e `runEvents`. Isso simplifica a sintaxe e, desde que a instância da mônada cumpra as três leis, faz com que a composição de transições monádicas seja associativa e tenha identidade (Bocchino, 2025, seção 7), pré-requisito para compor FSMs complexas em arquiteturas modulares.

2.4 Composabilidade Arquitetural e o Framework CREM

A validação final da abordagem reside na sua capacidade de suportar arquiteturas de *software* complexas. O CREM nasce de ideias do *Domain-Driven Design* (DDD), em que partes

do domínio de um sistema, como agregados, políticas e projeções, são modeladas como máquinas de estados (Perone e Karachalias, 2023, seção 2). Neste trabalho, o interesse recai sobre a composição dessas máquinas, e não sobre o DDD em si.

O *framework* Crème de la Crem (CREM) é um exemplo prático que demonstra o potencial máximo da PFP nesse contexto. O CREM utiliza os recursos avançados de *type-level* de Haskell para permitir que o *software* seja estruturado como uma composição de máquinas de estados independentes (Perone e Karachalias, 2023). Essa modularidade permite que subsistemas de domínio sejam desenvolvidos de forma isolada e testada, e combinados de forma segura.

A maior contribuição do CREM é a capacidade de codificar regras de negócio como tipos. Ele permite ao usuário especificar transições de estado permitidas e proibidas diretamente na assinatura de tipos (Perone e Karachalias, 2023). Isso significa que as restrições mais complexas da lógica de domínio da FSM são transformadas em restrições de tipo, e uma transição que viole a topologia resulta em erro de compilação, e não em erro de *runtime*; regras que dependem de sequências de eventos ou do evento que dispara a transição não são cobertas (seção 5.15). Além disso, o CREM é representável no sentido dos autores: a implementação de um sistema se mantém sincronizada com a sua representação gráfica, pois o diagrama é gerado a partir do próprio código (Perone e Karachalias, 2023). O termo não se refere ao functor representável da teoria das categorias.

3 PROCEDIMENTOS METODOLÓGICOS

Esta seção conserva a redação da proposta aprovada, no tempo verbal da época. Ao fim de cada fase, a frase "Situação" registra o que foi feito até agora.

A metodologia deste projeto será de natureza mista, combinando o rigor da pesquisa formal com a validação prática, pautando-se numa abordagem qualitativa para analisar a fundo o fenômeno da formalização de Máquinas de Estados Finitos (FSMs) no contexto da programação funcional pura. O trabalho se desenvolverá em duas vertentes principais: a Pesquisa Bibliográfica e a Pesquisa Aplicada (ou Tecnológica). A Pesquisa Bibliográfica fornecerá a base formal, coletando e analisando o referencial teórico sobre FSMs, Programação Funcional Pura (Haskell), Tipos de Dados Algébricos (ADTs), Teoria das Categorias (mônadas, Categoria Kleisli) e os princípios do *framework* CREM. Esta base teórica é crucial para a construção do modelo computacional.

A Pesquisa Aplicada será a fase de experimentação, focada no desenvolvimento de um artefato de *software* (código em Haskell) para comprovar as hipóteses de segurança e composabilidade. Os procedimentos serão organizados em quatro fases interligadas que visam, em última instância, cumprir todos os objetivos específicos do projeto.

A Fase 1: Revisão Teórica e Formalização Inicial visa estabelecer o alicerce do código. Inicialmente, será concluída a revisão aprofundada dos fundamentos da FSM e de como a imutabilidade da PFP se traduz numa alternativa superior ao modelo imperativo. Em seguida, o

primeiro objetivo específico será atendido pela modelagem de ADTs, onde os conjuntos de Estados (Q) e Eventos (Σ) de uma FSM canônica serão declarados como Tipos Soma em Haskell. O segundo objetivo será alcançado com a implementação da função de transição pura δ utilizando *pattern matching* exaustivo, o que é um procedimento metodológico chave para validar a Hipótese de Completude Funcional, demonstrando que o compilador verifica o tratamento de todas as combinações $(\text{Estado} \times \text{Evento})$. Situação: concluída quanto à modelagem e à função de transição; a revisão da literatura continua em andamento (seção 4).

A Fase 2: Gestão do Estado e Sequenciação Monádica endereçará o desafio da pureza na manipulação de estado, cumprindo o terceiro objetivo específico. O estudo da `State Monad` será traduzido na implementação, garantindo que a sequência de transições seja realizada através de uma função que calcula o novo estado a partir do antigo, mantendo a mudança de estado de forma pura (Milewski, 2020). Esta fase incluirá a sequenciação de eventos utilizando a notação *do-notation* de Haskell, demonstrando como a Categoria Kleisli fornece o formalismo necessário para a composição segura dessas transições monádicas. Situação: concluída (seção 5.7).

A Fase 3: Demonstração da Composabilidade e Segurança é a fase de validação prática da arquitetura. Será selecionado um Estudo de Caso de Domínio que exija a composição de múltiplas FSMs independentes. O procedimento metodológico aqui será compor as máquinas com os construtores do CREM (`Sequential`, `Parallel`, `Alternative` e `Feedback`), apoiados na composição de Kleisli, e verificar a modularidade arquitetural, simulando o conjunto em malha aberta e em malha fechada e validando o comportamento por enumeração limitada de cenários. O ponto de maior rigor será o Teste de Invalidez de Tipagem, que envolverá a tentativa intencional de codificar uma transição proibida pela lógica do domínio da FSM. A documentação do erro de compilação resultante comprovará a Hipótese de Segurança de Tipos, validando que o sistema de tipos de Haskell é capaz de especificar transições de estado permitidas e proibidas (Perone e Karachalias, 2023) em tempo de compilação. Situação: concluída para o teste de invalidez (16 pares, seção 5.12) e parcial para a composição (seções 5.10, 5.11 e 5.13).

Por fim, a Fase 4: Análise e Conclusão consistirá na comparação qualitativa dos resultados do modelo funcional puro implementado com o modelo imperativo tradicional, focando na Segurança de Tipos, Completude e Modularidade alcançadas. O código-fonte desenvolvido servirá como a evidência principal para a elaboração final do relatório, que sistematizará o conhecimento adquirido e apresentará a validação conclusiva das hipóteses de pesquisa. Situação: iniciada; a comparação com a abordagem imperativa foi feita em duas linguagens (C e C++), sem estudo de esforço de manutenção (seção 5.14).

4 ESTADO DA ARTE

Esta seção compara quatro maneiras de implementar uma máquina de estados finitos, usando apenas as fontes reunidas até aqui. Não se trata de uma revisão sistemática: as fontes permitem descrever e confrontar as abordagens, mas não afirmar que cobrem a fronteira atual do conhecimento. As lacunas estão listadas no final da seção.

4.1 O problema nas fontes

Malakhovski e Shalyto (2009) tratam de autômatos que reagem a eventos, em linguagens funcionais puras. Eles observam que, em linguagens imperativas, o estado costuma ficar em um campo de classe ou em uma variável global, o que não existe em uma linguagem pura. Observam também que os identificadores de estado costumam ser inteiros ou cadeias, às vezes agrupados em um `enum`, o que não oferece verificação satisfatória quando a função de transição é escrita à mão. Perone e Karachalias (2023) partem de outro incômodo, a distância entre o modelo do domínio e a implementação, comparável à de uma documentação desatualizada, e propõem construir arquiteturas como composição de máquinas de estados que possam ser representadas graficamente.

4.2 Quatro abordagens

FSM imperativa. O exemplo de Malakhovski e Shalyto (2009, p. 21) é um interruptor de lâmpada em C, com dois `enum`, uma variável global e uma função com `switch`. Os autores notam que nada impede, por exemplo, comparar o estado com um evento. A conferência fica com os testes em tempo de execução. Não há, na base, fonte que meça a taxa de erros dessa abordagem em comparação com as demais.

Haskell ingênuo. Na versão em Haskell do mesmo exemplo, estados e eventos são tipos algébricos e a função de transição é definida por equações. Os autores afirmam que o compilador confere tipos, completude e ausência de contradições; no exemplo deles, uma equação supérflua gera um aviso. A execução sobre uma lista de eventos é uma dobra à esquerda (`foldl`). Nos experimentos deste projeto (GHC 9.6.7), a omissão de um par produz um aviso que nomeia o par, e só se torna erro com `-Werror`. Além disso, uma equação curinga escapa dessa verificação, e o compilador confere se a tabela está completa, não se é a pretendida.

State Monad. Os mesmos autores constroem uma mônada própria para o autômato, com operações para ler e gravar o estado, de modo que uma sequência de eventos vira um bloco `do` (2009, pp. 21–22). Wadler (1992) mostra como mônadas permitem escrever em linguagem pura programas que manipulam estado, com ganho de legibilidade e menos erros do tipo passar o contador errado. Hutton (2016, cap. 12) apresenta a State Monad usada neste projeto. Do lado categorial, Milewski (2020, cap. 4) e Maia (2022) apresentam a composição de Kleisli, Bocchino (2025) relaciona as leis da mônada a essa composição e Jahn (2020) formaliza a

mônada como monoide na categoria dos endofuntores. O tipo do estado, porém, não diz quais transições são permitidas: qualquer função de estado em par (valor, novo estado) serve.

CREM. Perone e Karachalias (2023) definem a máquina de estados como máquina de Mealy, isto é, uma função com estado que, a cada entrada, emite uma saída e atualiza o estado. As transições permitidas formam uma topologia promovida ao nível dos tipos, e o resultado de cada ação só é aceito se o compilador encontrar a aresta correspondente. As máquinas são folhas de uma árvore, combinadas por construtores de composição. Os autores citam o pacote `motor`, que também gera diagramas, mas para máquinas isoladas, e apontam limitações: a topologia não se liga às entradas que disparam as transições, e a implementação não explora concorrência.

4.3 Quadro comparativo

	Imperativa	Haskell ingênuo	State Monad	CREM
Estado	campo ou variável global	argumento e resultado	dentro da computação	dentro da máquina; `run` devolve a máquina atualizada
Identificador de estado	inteiro, cadeia ou `enum`	tipo algébrico	tipo algébrico	tipo algébrico promovido a tipo
Equação ausente	testes	aviso do GHC; erro com `-Werror`	igual ao ingênuo	igual ao ingênuo
Aresta fora do modelo	testes	não detectada	não detectada	rejeitada na compilação
Evento e aresta	código	código	código	só no código
Composição	manual	manual	Kleisli, com leis a provar	construtores do CREM
Diagrama	não previsto	não previsto	não previsto	módulo `Render`

Tabela 2: Comparação das quatro abordagens. As linhas sobre avisos, rejeição de arestas e composição vêm de experimentos deste projeto; as demais, das fontes citadas no texto.

4.4 Lacunas

Faltam fontes, e portanto não há afirmação, sobre: artigos de conferência e periódicos que unam máquinas de estados, Haskell com tipos avançados e teoria das categorias; outras bibliotecas de máquinas de estados em Haskell além das que Perone e Karachalias apenas mencionam; tipos dependentes como alternativa; *statecharts*; verificação de modelos para propriedades de alcançabilidade (os autores do CREM citam TLA+ e Alloy apenas como trabalho futuro); comparações empíricas entre implementações imperativas e funcionais; e requisitos de segurança do domínio de aviônica, que aqui é apenas um domínio de exemplo escolhido pelo projeto. A fonte de Malakhovski e Shalyto está em russo; as páginas 20 e 21 foram lidas no original com apoio de uma ferramenta de inteligência artificial, sem revisão por falante nativo, e as paráfrases devem ser conferidas. O restante do levantamento será feito junto com o desenvolvimento, conforme o cronograma.

5 PROGRESSO PARCIAL

Esta seção apresenta os resultados obtidos até o momento, na ordem cronológica do trabalho, relacionando cada trecho de código ao referencial teórico. Os exemplos iniciais usam uma catraca (`stateMachine.hs` e `crem-example`); a partir da seção 5.8, o estudo de caso é a máquina de aviônica de um veículo aéreo não tripulado (UAV), implementada no projeto `avionics-crem` (repositório Git, versão `ab02fea`, etiqueta `v0.2.0-pic-parcial`; Apêndice A). Todo código e toda saída de compilador citados foram produzidos com GHC 9.6.7 e cabal 3.12.1.0, sobre o pacote `crem` 0.1.0.0.

5.1 Correspondência entre Tipos de Dados Algébricos e os Conjuntos Q e Sigma

O primeiro resultado é a tradução direta dos conjuntos finitos de estados (Q) e eventos (Σ) da quíntupla da FSM para tipos soma em Haskell, no arquivo `stateMachine.hs`:

```
data State = Locked | Unlocked deriving (Show, Eq)
data Input = Coin | Push deriving (Show, Eq)
```

Do ponto de vista categorial, essa codificação instancia um resultado conhecido. Milewski (2020, cap. 5) mostra que o tipo `Bool`, com dois construtores, corresponde ao número 2, entendido como o coproduto de duas cópias do objeto terminal ($2 = 1 + 1$). Coproduto é a construção que reúne dois tipos em um só, de modo que cada valor pertence a um ou ao outro; o objeto terminal é o tipo com um único valor. Assim, `State` e `Input` são isomorfos a `Bool`, na categoria Hask, cujos objetos são os tipos de Haskell e cujos morfismos são as funções entre eles. A finitude exigida para Q e Σ fica, desse modo, preservada pelo próprio tipo, e não apenas por convenção de nomes.

Uso no trabalho. A leitura categorial de coproduto e objeto terminal explica por que o tipo tem exatamente tantos valores quanto construtores, mas nenhum resultado posterior do projeto depende dela; é contexto, e não ferramenta.

5.2 A Função de Transição delta como Morfismo em Hask

O segundo resultado é a função de transição $\delta : Q \times \Sigma \to Q$, escrita por casamento de padrões:

```
transition :: State -> Input -> State
transition Locked   Coin  = Unlocked
transition Locked   Push  = Locked
transition Unlocked Coin  = Unlocked
transition Unlocked Push  = Locked
```

A assinatura `State -> Input -> State` é a forma *curried* de uma função cujo domínio é o par `(State, Input)`. *Currying* é a equivalência entre uma função de dois argumentos, `(a, b) -> c`, e uma função que recebe um argumento e devolve outra, `a -> b -> c`; Milewski (2020, cap. 9, seção 9.2) a explica e a liga ao objeto exponencial, e Hutton (2016, seção 3.6) trata o mesmo assunto como funções *curried*. O domínio é o produto categorial de `State` e `Input`, construído por Milewski (2020, cap. 5) para pares. A assinatura em Haskell corresponde, portanto, diretamente à definição matemática de δ.

Quanto à completude, o GHC examina as equações e avisa se algum par (estado, evento) ficou sem tratamento. Esse aviso (`-Wincomplete-patterns`, incluído em `-Wall`) não impede a compilação por si só; para isso é preciso promovê-lo a erro com `-Werror=incomplete-patterns`, como faz o projeto `avionics-crem`. Hutton (2016, seção 3.1) lembra que a inferência de tipos acontece antes da execução, de modo que erros de tipo não ocorrem durante a avaliação; a verificação de exaustividade é uma checagem adicional, também feita em compilação, mas de natureza diferente. Ela confere que cada par tem uma equação, não que a equação seja a correta: trocar `= Locked` por `= Unlocked` na segunda linha compila sem aviso.

5.3 Sequenciação Inicial e Validação Empírica

A composição de várias transições foi resolvida inicialmente com a função de ordem superior `foldl`, que aplica δ repetidamente sobre uma lista de entradas:

```
run :: State -> [Input] -> State
run = foldl transition
```

A execução sobre `[Coin, Push, Push, Coin, Coin, Push]`, a partir de `Locked`, produziu o estado final `Locked`. A função `transition` é pura no sentido de Milewski (2020): dado o mesmo argumento, devolve sempre o mesmo resultado e não tem efeitos colaterais. Essa pureza, somada à ausência de estado mutável, descrita por Malakhovski e Shalyto (2009), é o que permite aplicar a verificação de tipos e de exaustividade como mecanismos de corretude.

5.4 Da Composição Ingênua à Necessidade de uma Abstração Monádica

A composição por `foldl` basta enquanto o resultado de cada passo é só o novo estado. Quando se quer também uma saída por transição, por exemplo uma mensagem de telemetria, há duas formulações equivalentes.

A primeira não usa mônada. É a máquina de Mealy, em que cada passo tem o tipo $(Q, \Sigma) \to (Q, \text{Saída})$ e a execução sobre uma lista usa `mapAccumL`, que carrega o estado como acumulador e coleta as saídas. No projeto, essa é a função `runMealy`.

A segunda é a State Monad, em que o passo é uma função $\Sigma \to ST\ Q$ Saída. Nos testes do projeto, as duas formulações foram executadas sobre as mesmas sequências de eventos, a partir dos quatro estados iniciais, e produziram os mesmos pares (saídas, estado final). Logo, produzir saída por transição não exige mônada.

O que a formulação monádica acrescenta é composição. Milewski (2020, cap. 4) chama de funções embelezadas as funções do tipo $A \to M\ B$, em que M é um functor que carrega o efeito (um functor é um construtor de tipos que também sabe aplicar funções a seus conteúdos). A categoria de Kleisli tem os tipos como objetos, essas funções como morfismos e um operador `>=>`, chamado *fish*, para compô-las; Maia (2022) apresenta o mesmo desenvolvimento a partir do *Writer*. Com ele, um passo posterior pode depender da saída de um anterior sem que o estado seja passado à mão, e o mesmo molde serve para outros efeitos além do estado. A justificativa para a State Monad é, portanto, composição e generalidade, e não necessidade.

5.5 Reformulação Teórica da FSM sobre a State Monad

Para tratar do estado, e não apenas de um registro, a FSM foi mapeada sobre a State Monad de Hutton (2016, cap. 12): um `newtype` que embrulha uma função de estado em par (valor, novo estado). O projeto usa a mesma definição, com o tipo do estado como parâmetro:

```
newtype ST s a = ST { runST :: s -> (a, s) }

instance Monad (ST s) where
  st >>= f = ST $ \s ->
    let (x, s') = runST st s
     in runST (f x) s'
```

As instâncias de `Functor` e `Applicative` seguem Hutton e estão em `Avionics/StateMonad.hs`. Um passo da máquina de aviônica é então:

```
avionicsStep :: AvEvent -> ST AvState Output
avionicsStep e = ST $ \q -> (outputOf q e, transition q e)
```

Esse passo tem o mesmo molde do exemplo de Hutton (2016) para gerar valores frescos, `fresh = S (\n -> (n, n+1))`: um valor interno é passado de forma pura de uma aplicação à seguinte, sem mutabilidade.

5.6 Sequenciação via Notação do e a Função mapM

Para generalizar a composição de `foldl` ao contexto monádico, Hutton (2016, cap. 12) define `mapM`, que aplica uma função monádica a cada elemento de uma lista e reúne os resultados. No projeto, a função `runEvents` faz o mesmo para a lista de eventos:

```
runEvents :: [AvEvent] -> ST AvState [Output]
runEvents []       = pure []
runEvents (e : es) = do
  o  <- avionicsStep e
  os <- runEvents es
  pure (o : os)
```

A execução `runST (runEvents eventos) Grounded` devolve ao mesmo tempo a lista de saídas e o estado final. A notação `do` é uma forma de escrever expressões com o operador »= de maneira próxima à das linguagens imperativas, preservando a pureza (Oliveira, 2017).

5.7 Fundamentação Categorial: a Mônada como Monoide na Categoria dos Endofuntores

Um endofuntor é um functor de uma categoria nela mesma; em Haskell, é um construtor de tipos `m` com `fmap`. Uma mônada é um endofuntor T com duas transformações naturais, $\mu : T \circ T \to T$ (achatar) e $\eta : I \to T$ (injetar), cujas leis de associatividade e de unidade têm a forma das leis de um monoide. É a leitura "monoide na categoria dos endofuntores", presente em Milewski (2020, cap. 22) e em Jahn (2020, cap. 2).

A consequência prática é que a composição de Kleisli `>=>` é associativa e tem `return` como identidade. Isso, porém, só vale se a instância cumprir as leis, e o GHC não as verifica: uma instância que as violasse compilaria normalmente. Por isso as leis foram demonstradas para a instância `ST`, por raciocínio equacional (Hutton, 2016, seção 16.1; as leis aparecem na seção 12.3), em comentário no arquivo `Avionics/StateMonad.hs`. Em resumo:

- identidade à esquerda, `return a »= f = f a`: basta substituir as definições e simplificar o `let` sobre um par conhecido;
- identidade à direita, `m »= return = m`: o `let` devolve o par `(fst p, snd p)`, que é igual a `p`;
- associatividade, `(m »= f) »= g = m »= (\x -> f x »= g)`: expandem-se os dois lados e achatam-se os `let` aninhados, obtendo-se expressões idênticas.

As demonstrações supõem computações totais e ausência de `seq`, hipótese análoga à de Hutton (2016, seção 12.4). Bocchino (2025, seções 4.4 a 4.6 e 6.4) mostra que as três leis de Haskell equivalem à associatividade e às identidades da composição de Kleisli, e (seção 7) que, com elas, os tipos e as funções `a -> M b` formam uma categoria. Além da prova, as leis foram conferidas por exemplo, para os quatro estados e os quatro eventos (dezessete verificações em `lawChecks`: sete sobre mônada e Kleisli, oito sobre Functor e Applicative e duas de consistência com `runMealy` e `foldr (>=>)`; todas aprovadas).

As leis de Functor e de Applicative, que Hutton (2016, seções 12.1 e 12.2) enuncia à parte, também foram demonstradas para `ST`. Como as instâncias de `Functor` e `Applicative` foram escritas à mão, mostrou-se que `fmap f m` é igual a `m >>= pure . f` e que `mf <*> mx` é igual a `mf >>= (\f -> mx >>= pure . f)`; as quatro leis de Applicative decorrem dessa coincidência e das leis da mônada. A demonstração da lei de composição do Applicative está resumida, e é a menos detalhada das quatro.

Uso no trabalho. A leitura da mônada como monoide na categoria dos endofuntores é contexto: nenhum resultado do projeto depende dela. O que tem consequência no código é a composição de Kleisli, usada em `>=>`, em `composeAll` e nos testes de associatividade.

Uma observação sobre o CREM: no código do pacote, a função `pureResult` exige `Applicative m`, enquanto `run` e o sinônimo `StateMachine` exigem `Monad m`. `Applicative` é uma interface mais fraca que `Monad`, pois oferece `pure` e `<*>`, mas não `>>=`. O CREM é, portanto, parametrizado por um efeito `m`, o que não implica que a State Monad seja pré-requisito: o CREM nem a usa, e guarda o estado dentro da própria máquina, devolvendo a máquina atualizada a cada `run`. A relação com a seção anterior é de analogia, não de dependência.

5.8 Estudo de Caso: Topologia Promovida ao Nível de Tipos

O framework CREM (Perone e Karachalias, 2023) descreve as transições permitidas por uma topologia: para cada vértice, a lista dos vértices a que ele pode ir. A topologia é um valor comum de Haskell, depois promovido ao nível dos tipos com a biblioteca `singletons`. Um *singleton* é um tipo que dá, a cada valor de um tipo enumerado, um tipo correspondente no nível dos tipos, de modo que o compilador possa raciocinar sobre ele. A geração desses tipos é feita por Template Haskell, mecanismo de geração de código em compilação.

No estudo de caso de aviônica, os vértices e a topologia são declarados juntos:

```
$(singletons [d|
  data AvionicsVertex
    = Grounded | InFlight | EmergencyLanding | Shutdown
    deriving (Show, Eq)

  avionicsTopology :: Topology AvionicsVertex
  avionicsTopology = Topology
    [ (Grounded,          [InFlight, Shutdown])
    , (InFlight,          [EmergencyLanding])
    , (EmergencyLanding, [Grounded])
    ]
  |])
```

A regra de domínio é que um UAV em voo (`InFlight`) não pode ser desligado diretamente (`Shutdown`); por isso não existe essa aresta. A analogia com as classes de tipos de Hutton (2016), como `Eq` e `Ord`, vale em parte: ambas expressam restrições que o compilador resolve. A diferença é que uma classe restringe tipos, enquanto a topologia restringe valores (os vértices) promovidos a tipos pela biblioteca `singletons`, o que permite ao compilador raciocinar sobre eles.

A Figura 1 mostra as duas topologias do estudo de caso. As auto-transições (aresta identidade) não foram desenhadas.

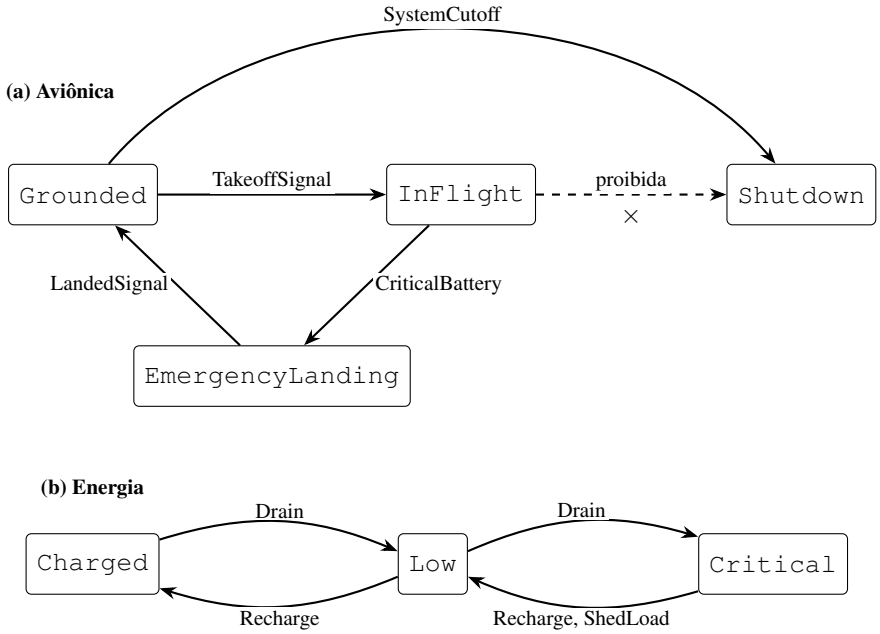


Figura 1: Topologias do estudo de caso. A aresta tracejada de `InFlight` a `Shutdown` não existe: é a transição proibida pela regra de domínio e rejeitada pelo compilador. `ShedLoad` foi acrescentado pelo simulador (seção 5.11) e usa a aresta `Critical` para `Low` que já existia.

A catraca, usada nas seções anteriores, tem a topologia com as arestas `Locked` para `Unlocked` e o inverso. O CREM admite sempre a aresta identidade (um vértice para ele mesmo), por uma instância da classe `AllowedTransition` (Perone e Karachalias, 2023, Fig. 4); logo, as auto-transições são sempre permitidas. Com dois vértices, as quatro arestas resultantes cobrem todos os pares, e a topologia da catraca é um grafo completo. Por isso ela não serve para demonstrar rejeição.

Também foram escritas, no mesmo projeto, a máquina `PowerFSM` (subsistema de energia, com os vértices `Charged`, `Low` e `Critical`) e as composições da seção 5.10.

5.9 Verificação Formal de Segurança de Tipos em Tempo de Compilação

As ações da máquina de aviônica têm a forma:

```
avionicsAction
```

```
  :: Applicative m
  => SAvionicsVertex v -> AvEvent
  -> ActionResult m AvionicsTopology SAvionicsVertex v Output
avionicsAction SGrounded TakeoffSignal =
  pureResult MsgTakeoff SInFlight
avionicsAction SInFlight CriticalBattery =
  pureResult MsgEmergency SEmergencyLanding
avionicsAction SInFlight TakeoffSignal =
  pureResult MsgIgnored SInFlight
-- ... (16 pares escritos um a um)
```

O primeiro argumento de `pureResult` é a saída; o segundo é o vértice de destino. A função exige a restrição `AllowedTransition topologia origem destino`, uma classe de tipos que o GHC resolve procurando a aresta na lista da topologia. Eventos que não provocam transição devolvem o mesmo vértice, o que a aresta identidade permite.

Para o teste de invalidez, o módulo `InvalidInFlightShutdown.hs` tenta a transição proibida, e fica fora do build principal (é um alvo de teste do cabal, construído com `cabal build invalid-inflight-shutdown -enable-tests`):

```
invalidInFlightShutdown SInFlight _ =
  pureResult MsgShutdown SShutdown
```

A compilação falha. Esta é a mensagem do GHC 9.6.7, sem edição, exceto pelo trecho central omitido, indicado por reticências (o texto integral está no arquivo `erro_invalidez_inflight_shutdown.txt`):

```
invalid\InvalidInFlightShutdown.hs:23:39: error: [GHC-39999]
    * Could not deduce ‘Crem.Topology.AllowedTransition
                         (Crem.Topology.Topology '[])
                         InFlight Shutdown’
        arising from a use of ‘pureResult’
      from the context: Applicative m
        bound by the type signature for:
                   invalidInFlightShutdown :: ...
      or from: ()
        bound by a pattern with constructor:
                   SInFlight :: SAvionicsVertex InFlight,
                 in an equation for ‘invalidInFlightShutdown’
    * In the expression: pureResult MsgShutdown SShutdown
      In an equation for ‘invalidInFlightShutdown’:
          invalidInFlightShutdown SInFlight _
```

```
= pureResult MsgShutdown SShutdown
```

O trecho `Topology '[]` mostra como a busca termina: as instâncias de `AllowedTransition` descartam as arestas da topologia uma a uma e, ao chegar à lista vazia sem encontrar `InFlight` para `Shutdown`, o GHC fica sem evidência e rejeita a expressão. A regra de negócio foi, portanto, verificada antes de o programa existir. Isso estende o princípio de segurança de tipos de Hutton (2016) do nível sintático para o nível do domínio, que é a proposta de Perone e Karachalias (2023) de especificar transições permitidas e proibidas no tipo.

Há um limite que precisa constar. A garantia vale para uma transição isolada. O caminho `InFlight` para `EmergencyLanding` para `Grounded` para `Shutdown` existe na topologia, e é aceito. A regra de que o UAV não se desliga em voo vale para a aresta direta, mas não é propriedade de todas as sequências de eventos; provar propriedades desse tipo exigiria outra técnica.

Esse teste exercita um único par de vértices; a matriz completa, com os 16 pares, está na seção 5.12.

Como o GHC resolve a restrição. No código do pacote `crem` 0.1.0.0, a classe `AllowedTransition` tem quatro instâncias, todas marcadas como incoerentes (`INCOHERENT`): uma encontra a aresta quando ela é a primeira da lista do vértice; outra descarta a primeira aresta do vértice e continua; outra descarta o primeiro vértice da topologia e continua; e a última, de aresta identidade, vale quando origem e destino são o mesmo vértice. A marcação de incoerência faz o GHC aceitar uma instância que casa sem exigir que seja a mais específica, o que torna a resolução uma busca por qualquer caminho que feche a prova. Se nenhum caminho fecha, sobra a restrição residual com a lista vazia, que é o `Topology '[]` da mensagem acima. A correção geral dessa resolução é responsabilidade da biblioteca e não foi provada aqui; verificou-se que, para esta topologia, o resultado coincide com a especificação nos 16 pares (seção 5.12).

5.10 Execução, Validação Cruzada e Composição

A execução do projeto (`cabal run avionics-crem`) sobre a sequência de eventos de decolagem, bateria crítica, pouso e corte do sistema produziu a mesma lista de saídas, `[MsgTakeoff, MsgEmergency, MsgLanded, MsgShutdown]`, e o mesmo estado final, `Shutdown`, nas três formulações: ingênua, State Monad e CREM. Além disso, as versões ingênua e CREM foram comparadas em todas as 85 sequências de até três eventos a partir de `Grounded`, e as saídas coincidiram em todas. Essa comparação é uma validação cruzada entre formalizações estruturalmente distintas da mesma máquina, mas não é independente quanto à tabela, que foi escrita uma só vez; a seção 5.12 a confronta com uma especificação separada.

A composição foi realizada com os construtores do CREM, em `Avionics/Compose.hs`. Cada folha (`Basic`) mantém seu próprio estado e sua

própria topologia.

- `Parallel`: aviônica e energia rodam lado a lado, cada uma com sua entrada e sua saída. Na execução, a entrada `(TakeoffSignal, Drain)` produziu `(MsgTakeoff, ReportLow)`.
- `Sequential` com `Alternative`: a saída de energia passa por uma máquina sem estado que a traduz em evento de aviônica, e a aviônica só é acionada quando há evento. Com a aviônica iniciada em `InFlight` e a bateria em `Charged`, o segundo `Drain` levou a bateria ao nível crítico, gerou o evento `CriticalBattery` e produziu a saída `MsgEmergency`, isto é, a aviônica passou a `EmergencyLanding`.

O construtor `Kleisli` não foi exercitado. O `Feedback` é usado no simulador da seção 5.11.

Os diagramas foram gerados com o módulo `Render` do CREM, em formato Mermaid (linguagem textual para diagramas): o das topologias de aviônica e de energia e o do fluxo da composição acoplada. Os arquivos estão em `Obsidian/documentation/` (`avionics_topology.mmd`, `power_topology.mmd`, `coupled_flow.mmd`). Conferiu-se o texto gerado; os diagramas não foram renderizados por uma ferramenta Mermaid.

5.11 Simulador em Malha Aberta e Fechada e Robustez dos Testes

Até a seção 5.10, a verificação de comportamento se apoiava em uma sequência fixa de eventos e em 85 sequências curtas. Para tornar o teste mais robusto, foi escrito um simulador em Haskell puro da composição de máquinas (módulos `Avionics.Sim.*`, sem IO, sem estado mutável e sem aleatoriedade), com uma suíte de testes que percorre todos os cenários até um comprimento dado. Ele valida o comportamento discreto das máquinas e da composição; não modela a física de voo.

Alguns termos. Um *tick* é um passo discreto de simulação, sem correspondência com segundos. Um *traço* é o registro, por tick, da entrada recebida e dos passos de cada máquina (evento, saída e vértice em que ficou). Em malha aberta, cada máquina só recebe o que o cenário manda. Em malha fechada, o ambiente observa a telemetria e realimenta as máquinas, de modo que a saída de uma influencia a entrada da outra.

Arquitetura. A planta é a composição em paralelo da aviônica e da energia, com as ações `avionicsAction` e `powerAction` originais. O ambiente é uma terceira máquina, `Mission`, com quatro vértices (`Standby`, `CriticalHeld`, `TouchdownHeld` e `CutoffHeld`), topologia tipada e 20 equações, sem curinga. Ela é ligada à planta pelo construtor `Feedback` do CREM: o caminho direto contém a planta e o ambiente, e o caminho de retorno reinjeta os sinais que o ambiente liberou. A State Monad do projeto carrega, de passo em passo, a própria máquina composta como estado. Para relatar em que vértice cada máquina

ficou, o simulador observa o vértice de destino dentro da ação, sem reescrever nenhuma regra de transição; o resultado é reconstruído com a mesma prova `AllowedTransition`, e a garantia por aresta continua valendo. A Figura 2 resume o laço.

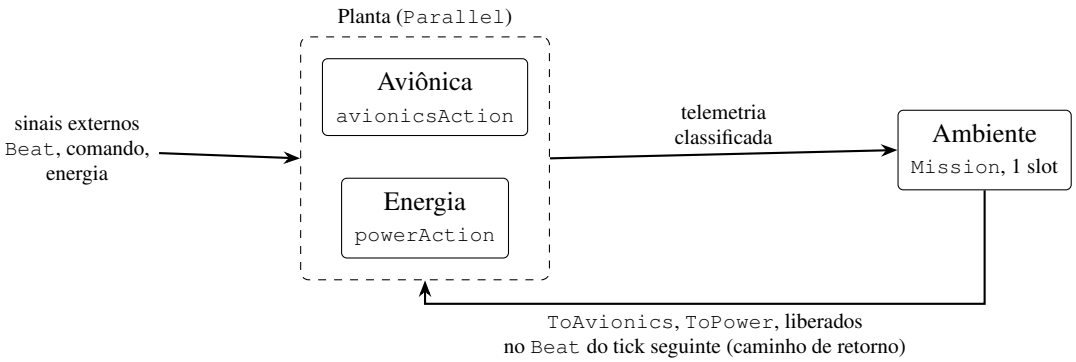


Figura 2: Laço fechado com o `Feedback` do CREM. O ambiente retém a telemetria relevante e só a libera no `Beat` do tick seguinte, o que produz o atraso de um tick.

Duas decisões de modelagem. Primeira, o `Feedback` do CREM é instantâneo: o laço é repetido dentro do mesmo tick até o caminho de retorno parar de emitir. O efeito desejado era influenciar o tick seguinte; ele foi obtido no próprio modelo, com a `Mission` funcionando como um registrador de um slot: a telemetria relevante a faz reter um evento, e o sinal de relógio (`Beat`) do tick seguinte a faz liberá-lo. Como o `Beat` vem só do cenário e a máquina só emite ao recebê-lo, o laço termina; esse argumento está no código, e não há guarda de terminação em execução. Segunda, para que a emergência também afete a energia, foi acrescentado um evento `ShedLoad` (corte de cargas não essenciais) a `PowerEvent`. Ele usa apenas a aresta `Critical` para `Low`, que já existia, de modo que a topologia de energia não mudou. O atraso de um tick, o próprio `ShedLoad` e a ordem em que o ambiente libera `ShedLoad` e `LandedSignal` são escolhas de modelagem do autor, sem justificativa em requisitos de um domínio real (seção 5.15).

Resultado de compilação. `cabal build`, com todos os avisos ligados e a exaustividade promovida a erro, terminou com código 0 e sem avisos do GHC. O teste de invalidez continua falhando com a mesma mensagem (`Could not deduce AllowedTransition` para `InFlight` e `Shutdown`, código de saída 1).

Traço em malha fechada, cenário de emergência, obtido com `cabal run avionics-sim`, na Tabela 2.

O mesmo cenário em malha aberta termina com a bateria em `Critical` e a aviônica ainda em `InFlight`: sem realimentação, nada liga uma máquina à outra. Em malha fechada, a bateria termina em `Low`, porque a emergência dispara o `ShedLoad`.

Suíte de testes. O comando `cabal test avionics-sim-tests` (comprimento máximo $N = 4$) executou 32 verificações, com 0 falhas. Os cenários de validação cruzada são as sequências de até 4 comandos, isto é, $1 + 4 + 16 + 64 + 256 = 341$; nas verificações com o alfabeto completo de entradas de um tick (20 símbolos), são $1 + 20 + 20^2 + 20^3 + 20^4 = 168\,421$

Tick	Entrada ou liberação	Aviônica	Energia	Ambiente
0	`TakeoffSignal`	`MsgTakeoff`, vai a `InFlight`	`Charged`	`Standby`
1	`Drain`	`InFlight`	`ReportLow`, `Low`	`Standby`
2	`Drain`	`InFlight`	`ReportCritical`, `Critical`	retém `CriticalHeld`
3	libera `CriticalBattery`	`MsgEmergency`, vai a `EmergencyLanding`	`Critical`	retém `TouchdownHeld`
4	libera `ShedLoad` e `LandedSignal`	`MsgLanded`, vai a `Grounded`	`ReportLow`, `Low`	retém `CutoffHeld`
5	libera `SystemCutoff`	`MsgShutdown`, vai a `Shutdown`	`Low`	`Standby`

Tabela 3: Traço do cenário de emergência em malha fechada. As saídas da aviônica são, em ordem, as mesmas quatro da execução em malha aberta.

cenários.

Verificação	Cenários	Tempo
malha aberta igual à FSM ingênua (saídas e estado final)	341	0,0 s
malha fechada igual à FSM ingênua, com os eventos que a aviônica recebeu de fato	341 e 168 421	0,01 s e 7,5 s
planta observada igual a `uavSystem` (Seção 5.10)	22 621	0,15 s
`Shutdown` só depois de `Grounded` (aberta e fechada)	168 421	0,5 s e 7,5 s
`EmergencyLanding` só a partir de `InFlight` (aberta e fechada)	168 421	0,5 s e 7,4 s
`ShedLoad` nunca reduz o nível da bateria (aberta e fechada)	168 421	0,5 s e 7,4 s
determinismo (aberta e fechada)	168 421	0,7 s e 8,5 s
causalidade: o traço de um prefixo é prefixo do traço (aberta e fechada)	8 421	0,08 s e 0,9 s
o `Beat` libera exatamente o que foi retido no tick anterior	168 421	7,4 s

Tabela 4: Verificações da suíte (execução desta revisão; os tempos valem como ordem de grandeza). Em malha fechada compara-se a FSM ingênua com os eventos efetivamente recebidos pela aviônica, porque o ambiente injeta eventos extras.

Sanidade das próprias verificações. Uma propriedade que nunca falha pode ser vacuamente verdadeira. Por isso a suíte inclui um bloco que as aplica a traços corrompidos à mão (por exemplo, um `Shutdown` vindo de `InFlight`), e as sete verificações desse bloco confirmam que os defeitos são detectados e que os casos válidos são aceitos. Além disso, nesta revisão foi feito um teste de mutação: trocar, em `Mission`, o evento liberado por `CriticalHeld` (`CriticalBattery` por `LandedSignal`) fez 6 das 32 verificações falharem; o arquivo foi restaurado em seguida pelo controle de versão. A documentação do repositório registra mutações anteriores, feitas com uma versão anterior da suíte (27 verificações), nas quais 4 e 9 verificações falharam; essas não foram repetidas aqui.

5.12 Conformidade com a Especificação do Domínio e Matriz de Rejeição

As validações anteriores tinham duas fraquezas. Primeira, a FSM ingênua, as ações do CREM e o ambiente do simulador foram escritos a partir da mesma tabela, de modo que compará-los entre si mostra que o CREM executa a tabela, não que a tabela está certa. Segunda, a rejeição em compilação era exercitada em um único par de vértices.

Especificação separada. As regras do domínio foram escritas em um arquivo próprio, `spec/domain_rules.txt`, a partir do texto do enunciado do estudo de caso e não do código: quatro arestas permitidas, cada uma com o evento que a dispara (`Grounded` para `InFlight` por `TakeoffSignal`; `InFlight` para `EmergencyLanding` por `CriticalBattery`; `EmergencyLanding` para `Grounded` por `LandedSignal`; `Grounded` para `Shutdown` por `SystemCutoff`) e uma proibição explícita (`InFlight` para `Shutdown`). Vale a convenção de mundo fechado: o que não é permitido nem identidade é proibido. Duas escolhas não vêm do enunciado e estão marcadas no arquivo: aceitar a aresta identidade, e deixar o estado inalterado nos pares (estado, evento) sem regra. A especificação foi escrita pela mesma pessoa que escreveu o código; é um artefato distinto, derivado de um texto anterior, mas não é um oráculo de terceiros.

Teste exaustivo. O comando `cabal test avionics-spec-tests` executa 16 verificações sobre todo o domínio (4 vértices e 4 eventos, sem amostragem). Elas confrontam a especificação com (A) o valor da topologia do CREM, (B) a função de transição ingênua, (C) o destino que a ação do CREM realmente devolve, lido do tipo singleton do resultado, e (D) a associação de cada evento ao destino previsto no enunciado. Todas passam.

Matriz de rejeição. O programa `scripts/negative_matrix.sh` gera um módulo mínimo para cada um dos 16 pares (origem, destino), com `act SOrigem = pureResult MsgIgnored SDestino`, e o compila. O esperado, segundo a especificação, é que compilem as quatro arestas e as quatro identidades, e que os outros oito pares sejam rejeitados com `Could not deduce AllowedTransition` nomeando o par e nenhum outro erro. Resultado: 8 compilaram, 8 foram rejeitados, 0 divergências.

origem \ destino	`Grounded`	`InFlight`	`EmergencyLanding`	`Shutdown`
`Grounded`	compila (identidade)	compila	rejeitado	compila
`InFlight`	rejeitado	compila (identidade)	compila	rejeitado
`EmergencyLanding`	compila	rejeitado	compila (identidade)	rejeitado
`Shutdown`	rejeitado	rejeitado	rejeitado	compila (identidade)

Tabela 5: Resultado da compilação de `pureResult` para os 16 pares de vértices (GHC 9.6.7).

As verificações conseguem falhar? Defeitos foram injetados um a um, e o arquivo restaurado pelo controle de versão após cada teste. A aresta `InFlight` para `Shutdown` acrescentada à topologia e à ação fez o teste de especificação falhar em quatro verificações (versão de 13 verificações do teste) e a matriz acusar uma divergência: o par passou a compilar. Uma alteração só na FSM ingênua (`InFlight` com `SystemCutoff` indo a `Shutdown`) fez o teste falhar em três verificações e a matriz continuou passando, como esperado, porque ela não olha a FSM ingênua. Uma ação do CREM com destino errado em uma aresta permitida (`Grounded` com `TakeoffSignal` indo a `Shutdown`) é aceita pelo compilador e pela matriz; só o teste de especificação a detecta, e apenas por causa do bloco (D), que associa o evento ao destino. Esse último caso mostra o limite da garantia dos tipos: o compilador aceita qualquer destino que seja aresta da topologia.

5.13 Cadeia da Bateria e Descarte do Registrador de um Slot

A suíte do simulador, com comprimento máximo $N = 4$, não alcança a cadeia de emergência induzida pela bateria (decolagem, dois `Drain` e três ticks de liberação, seis ticks no total). Foi escrita uma enumeração em malha fechada sobre um alfabeto reduzido de quatro ticks (ocioso, `TakeoffSignal`, `Drain` e `Recharge`) com $N = 8$, isto é, 87 381 cenários, executada em cerca de 33 s (`cabal test avionics-chain-tests`).

Resultado	Valor
invariante: toda entrada da bateria em `Critical`, com a aviônica em voo, leva a `EmergencyLanding` no tick seguinte	vale nos 87 381 cenários
cenários que alcançam `EmergencyLanding`	22 172
cenários que terminam em `Shutdown` tendo recebido ao menos um `Drain`	8 160
achado: terminam com a bateria em `Critical`, a aviônica em voo e dois ticks ociosos	172
achado: cenários em que o ambiente descartou telemetria	8 726 de 87 381

Tabela 6: Enumeração da cadeia da bateria (malha fechada, $N = 8$).

O primeiro achado é uma lacuna do modelo do ambiente, não do CREM. A máquina `Mission` reage à borda *a bateria entrou em nível crítico*, e não ao nível *a bateria está em nível crítico*. O menor exemplo é `Drain, Drain, TakeoffSignal, ocioso, ocioso`: a bateria chega a `Critical` em solo, o sinal é consumido ali (a aviônica o ignora), e a decolagem posterior ocorre com a bateria já crítica, sem que nada leve a aviônica à emergência. A `Mission` não foi alterada: reagir ao nível é uma decisão de modelagem que cabe ao autor; a política alternativa foi avaliada em um modelo de referência (abaixo).

O descarte de telemetria ocorre só quando o ambiente está retendo um evento de pouso ou de corte (`TouchdownHeld` ou `CutoffHeld`) e chega a notícia de bateria crítica (5 651 e 4 144 ocorrências). Pela estrutura do modelo, o ambiente só retém esses dois eventos depois de a aviônica ter passado por emergência ou pouso, de modo que a notícia perdida diz respeito a uma bateria cuja emergência já foi tratada; o efeito do descarte sobre o estado final foi medido com o modelo de referência descrito a seguir. Uma nova decolagem com a bateria ainda crítica recai na lacuna descrita acima. Fila ou prioridade são alternativas não avaliadas.

Modelo de referência independente e políticas do ambiente. Para que o ambiente não seja validado só contra o próprio simulador, o programa `cabal test avionics-policy-tests` reimplementa o laço fechado em Haskell comum, sem o CREM: a planta usa a função de transição ingênua e uma tabela de energia escrita à parte, e o ambiente segue a descrição do laço. Nos mesmos 87 381 cenários, esse modelo coincidiu com o simulador do CREM nos vértices de cada tick e nas saídas da aviônica; uma mutação (`CriticalHeld` liberando `LandedSignal`) faz a verificação falhar. O modelo foi escrito pela mesma pessoa que escreveu o simulador, mas a implementação não passa pelo CREM.

Com o mesmo modelo, três políticas para o ambiente foram comparadas: a do projeto (um slot, quem chega primeiro fica), uma fila sem perda (cada tipo de retenção entra uma vez) e uma

política que, além disso, reage ao nível da bateria (a cada `Beat`, com o slot livre, bateria em `Critical` e aviônica em voo, libera `CriticalBattery`).

Política	Estado final diferente da política do projeto	Terminam com bateria `Critical`, aviônica em voo e dois ticks ociosos
um slot (do projeto)	—	172
fila sem perda	0 de 87 381	172
reação ao nível	5 841 de 87 381	0

Tabela 7: Comparação de políticas do ambiente no modelo de referência (malha fechada, $N = 8$).

A fila sem perda não mudou nenhum estado final: a notícia descartada (bateria crítica durante o pouso ou o corte) chegaria a uma aviônica que já não está em voo, e `CriticalBattery` é ignorado fora de `InFlight`. Portanto, no recorte enumerado, o descarte do slot é inócuo. Já a lacuna entre borda e nível só desaparece com a reação ao nível (172 para 0 cenários), ao custo de mudar o estado final em 5 841 cenários, que são, em princípio, aqueles em que a aviônica está em voo com a bateria crítica sem que a política do projeto reaja; essa caracterização dos 5 841 casos não foi verificada caso a caso. Essas políticas existem só no modelo de referência; a `Mission` do projeto não foi alterada, e a escolha entre elas depende de requisitos que o projeto não tem.

A invariante consegue falhar? Ao fazer `CriticalHeld` deixar de liberar `CriticalBattery`, a invariante falhou com contraexemplo e o número de cenários que alcançam `EmergencyLanding` caiu a zero; o arquivo foi restaurado em seguida.

5.14 Comparação com a Mesma Máquina em C

Para avaliar a premissa do trabalho, a mesma máquina foi escrita em C (tipos `enum` e instruções `switch`, o estilo descrito por Malakhovski e Shalyto, 2009), e compilada com o *clang* 14.0.6, que acompanha o GHC. Sete variantes (M0 a M6) foram produzidas, cada uma com um defeito, e comparadas com a mesma especificação por um programa que imprime os 16 pares (estado, evento, destino). O compilador foi usado com dois conjuntos de opções: A (`-Wall -Wextra`) e B (A mais `-Wswitch-enum`, `-Wenum-compare` e `-Wassign-enum`).

A mesma comparação foi repetida em C++ com `enum class`, compilado com o *clang*++ da mesma distribuição. Os resultados para M1 a M4 coincidem com os de C. Muda M5: comparar o estado com uma constante de evento não compila em C++ com `enum class`, como em Haskell, ao passo que em C o `enum` comum aceita a comparação sem aviso. M6 (converter o valor 42 para o tipo do estado) compila sem aviso em C++, por meio de `static_cast`, e é impedido apenas em Haskell.

O que a comparação sustenta. Primeiro, o compilador de C, com as opções adequadas, também avisa de caso omitido; a diferença, nesse ponto, é que o aviso do GHC já está em `-Wall`, enquanto em C a variante com `default` só é acusada com `-Wswitch-enum`. Segundo, em

Defeito	C, opções A	C, opções B	Teste exaustivo em C	Haskell com CREM
M1: caso omitido no `switch`	1 aviso	1 aviso	passa (o retorno final equivale a não mudar de estado)	aviso do GHC; erro com a opção de promoção
M2: caso omitido e `default`	sem aviso	1 aviso	passa	sem aviso: o curinga esconde
M3: transição proibida `InFlight` para `Shutdown`	sem aviso	sem aviso	detecta	rejeitada na compilação
M4: aresta permitida, destino errado	sem aviso	sem aviso	detecta	aceita pelo compilador; o teste de especificação detecta
M5: compara estado com constante de evento	sem aviso	sem aviso	detecta só por coincidência numérica	erro de tipo
M6: valor fora da enumeração (42)	sem aviso	sem aviso	detecta	erro de tipo

Tabela 8: Defeitos injetados na máquina em C e o que cada mecanismo detecta.

domínio finito, um teste exaustivo detecta os mesmos defeitos M3 a M6 que o sistema de tipos; o que o CREM acrescenta, para M3, é detectá-lo na compilação, sem que seja preciso escrever o teste. Terceiro, nenhum compilador detecta M4, o destino errado em aresta permitida; é preciso uma especificação com eventos. Quarto, M5 é impedido pelo sistema de tipos de Haskell e pelo `enum class` de C++, mas não pelo `enum` de C, e M6 só é impedido em Haskell. Esses resultados dão forma mensurável à hipótese central: a vantagem é antecipação e ausência de código de teste, não uma garantia que um teste exaustivo não ofereça.

Limites. Foi comparada uma só máquina, em duas linguagens imperativas (C e C++) e com um só compilador (*clang*). Não se mediu esforço de escrita ou de manutenção, nem o custo dos módulos de Template Haskell e das mensagens de erro. A comparação com outras linguagens imperativas fica para a continuidade do trabalho.

5.15 Limitações Identificadas e Encaminhamentos

Registram-se as seguintes limitações.

Primeira, a garantia dos tipos é por aresta (seção 5.9). O caminho `InFlight`, `EmergencyLanding`, `Grounded`, `Shutdown` é aceito, aresta a aresta. As propriedades de sequência são verificadas por enumeração limitada (seções 5.11 e 5.13), e ausência de contraexemplo nesse recorte não é prova.

Segunda, o evento não faz parte do vértice. A topologia restringe quais vértices podem suceder quais, e não qual evento causa cada transição; essa relação fica no casamento de padrões da ação e é conferida, por teste, contra a especificação (seção 5.12). Perone e Karachalias (2023, seção 6) registram a mesma limitação como trabalho futuro.

Terceira, a especificação do domínio e o código foram escritos pela mesma pessoa. A especificação é um artefato derivado de um texto anterior, não de terceiros, e cobre apenas o que o enunciado diz; os pares sem regra seguem uma convenção do projeto. O mesmo vale para o modelo de referência da seção 5.13.

Quarta, o simulador é discreto: não há física, tempo contínuo nem duração de tick. A semântica do ambiente (bateria crítica vira sensor, emergência vira toque no solo, pouso vira corte), o atraso de um tick, o evento `ShedLoad` e a ordem de liberação entre `ShedLoad` e

`LandedSignal` são escolhas de modelagem do autor, sem justificativa em requisitos de um domínio real. Elas foram validadas contra a FSM ingênua e contra propriedades descritas, não contra requisitos de aviônica.

Quinta, a enumeração da seção 5.13 revelou que o ambiente reage à borda, e não ao nível, da bateria crítica. O modelo do projeto não foi alterado; no modelo de referência, a política que reage ao nível elimina o achado (172 para 0 cenários) e muda o estado final em 5 841 cenários. O descarte do slot não alterou nenhum estado final no recorte de 87 381 cenários, mas esse recorte usa quatro tipos de tick e $N = 8$.

Sexta, o laço não tem guarda de terminação em execução. Depende do argumento descrito na seção 5.11, e uma edição futura de `Mission` que emitisse em resposta a um sinal realimentado poderia travar o `Feedback`.

Sétima, a resolução da restrição `AllowedTransition` depende de instâncias marcadas como incoerentes (seção 5.9). Sua correção geral é responsabilidade da biblioteca e não foi provada aqui; verificou-se que, para esta topologia, o resultado coincide com a especificação nos 16 pares.

Oitava, o termo "representável". No título e no resumo de Perone e Karachalias (2023), uma máquina representável é aquela cuja implementação se mantém sincronizada com sua representação gráfica, da qual se gera um diagrama automaticamente. Isso é diferente do functor representável de Milewski (2020, cap. 14): um functor isomorfo ao functor de morfismos $\mathrm{Hom}(a, -)$, isto é, um contêiner indexado por chaves do tipo a. A ideia de usar o functor representável para indexar a topologia pelo alfabeto Σ é uma hipótese de trabalho do projeto, sem sustentação nas fontes da base.

Nona, o determinismo é trivial em código puro e a verificação existe como regressão; a causalidade é a verificação mais informativa dessa família. Os tempos citados são de uma máquina e de uma execução. Os diagramas Mermaid gerados nesta revisão não foram renderizados por uma ferramenta Mermaid.

Décima, a comparação com a abordagem imperativa (seção 5.14) cobre uma máquina, duas linguagens imperativas e um compilador. O estudo de esforço de manutenção, o custo do Template Haskell e a leitura das mensagens de erro não foram avaliados. O construtor `Kleisli` do CREM não foi exercitado. A literatura que falta (seção 4) permanece como lacuna.

5.16 Síntese Geral e Validação Preliminar das Hipóteses

Na implementação ingênua, a verificação de corretude é a exaustividade do casamento de padrões, que o GHC executa como aviso e que o projeto promove a erro. Na etapa monádica, a State Monad e a composição de Kleisli organizam a passagem de estado e saída, e as leis de mônada, Functor e Applicative foram demonstradas para a instância usada. Na etapa do CREM, a prova `AllowedTransition` é exigida pelo compilador, e uma transição fora da topologia é rejeitada em compilação, o que foi testado nos 16 pares de vértices. A composição foi realizada com `Parallel`, `Sequential`, `Alternative` e `Feedback`, e o conjunto foi

simulado em malha aberta e fechada.

A Tabela 9 separa o que foi demonstrado, o que o compilador rejeita e o que foi testado até um comprimento limitado.

Afirmação	Demonstrado	Rejeitado em compilação	Testado até comprimento limitado ou por enumeração
Leis de mônada, Functor e Applicative para `ST`	prova equacional, com hipóteses explícitas (sem `seq`, computações totais); a de composição do Applicative está resumida	—	17 verificações em exemplos (4 estados, 4 eventos)
Completude de δ e das ações	—	só se o aviso for promovido a erro; sem isso, apenas aviso	16 pares escritos um a um, sem curinga
Transição fora da topologia	—	8 dos 16 pares de vértices rejeitados; 8 compilam	teste de especificação, exaustivo no domínio de 16 pares
Destino correto de cada evento	—	não detectado pelo compilador	teste de especificação (bloco D)
Propriedades de sequência (`Shutdown` só depois de `Grounded`, etc.)	—	não aplicável	$N = 4$ (168 421 cenários); cadeia da bateria e ambiente conferido contra um modelo de referência sem CREM até $N = 8$ em alfabeto reduzido (87 381)
Composição preserva as garantias de cada máquina	não formalizado	por construção, em cada folha	observada em malha aberta e fechada

Tabela 9: O que cada tipo de evidência sustenta.

Quanto às hipóteses:

- Completude funcional: confirmada para as funções de transição e de ação da aviônica, da energia e do ambiente, escritas caso a caso e sem equação curinga, com a ressalva de que depende de promover o aviso a erro.

- Segurança de tipos: reformulada como *rejeição antecipada, em compilação, de transições fora da topologia*. Confirmada nesse sentido, nos 16 pares. A comparação com C mostra que um teste exaustivo detecta os mesmos defeitos em domínio finito; a vantagem observada é a antecipação e a ausência de código de teste. Não cobre sequências de transições nem a associação evento-destino.

- Composabilidade e modulação: demonstrada para `Basic`, `Sequential`, `Parallel`, `Alternative` e `Feedback`. Falta `Kleisli`. A propriedade de que a composição preserva as garantias de cada folha foi observada, não formalizada.

6 CRONOGRAMA

O cronograma a seguir descreve o planejamento de todas as etapas da pesquisa ao longo do período de desenvolvimento do projeto de Iniciação Científica.

Etapas	**fev**	**mar**	**abr**	**mai**	**jun**	**jul**	**ago**	**set**	**out**	**nov**	**dez**
Revisão bibliográfica aprofundada	X	X									
Modelagem de ADTs e função de transição		X	X								
Implementação de State Monad e Kleisli			X	X							
Estudo de caso e composição de FSMs				X	X						
Testes de invalidez de tipagem					X	X					
Análise comparativa e documentação						X	X				
Elaboração do relatório final							X	X	X		
Entrega e apresentação										X	X

Situação das etapas

Etapa	Situação	Observação
Revisão bibliográfica aprofundada	em andamento	lacunas listadas na seção 4
Modelagem de ADTs e função de transição	concluída	seções 5.1 e 5.2
Implementação de State Monad e Kleisli	concluída	leis demonstradas (seção 5.7)
Estudo de caso e composição de FSMs	em parte	falta o construtor `Kleisli`; seções 5.10 e 5.11
Testes de invalidez de tipagem	concluída	matriz de 16 pares (seção 5.12)
Análise comparativa e documentação	em andamento	comparação com C feita (seção 5.14); sem estudo de manutenção
Elaboração do relatório final	não iniciada	—
Entrega e apresentação	não iniciada	—

Tabela 10: Situação de cada etapa do cronograma.

DECLARAÇÕES

Disponibilidade de dados e código. Todo o código, os testes, a especificação do domínio e os arquivos de resultado estão no repositório descrito no Apêndice A. Não há conjunto de dados externo: os cenários são gerados por enumeração.

Ética. O trabalho não envolve seres humanos nem dados pessoais.

Contribuições dos autores (CRediT). Autor: Peterson Carara Junior. Orientação: Prof. Felipe Canarozzo Lourenço. *[A confirmar pelo autor: papéis de cada pessoa na taxonomia CRediT.]*

Conflito de interesses. [A confirmar pelo autor.]

Financiamento. [A preencher pelo autor: bolsa, agência e número do processo, ou "sem financiamento".]

Uso de ferramentas de inteligência artificial. O código, os testes, os experimentos e a redação deste relatório foram desenvolvidos com assistência do modelo Claude, da Anthropic, usado por meio do Claude Code. Isso inclui a leitura das páginas 20 e 21 de Malakhovski e Shalyto (2009), em russo, sem revisão por falante nativo. *[A confirmar pelo autor: o grau de revisão do texto e do código por ele, e a conformidade com as regras do programa de iniciação científica sobre esse uso.]*

7 REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS - ABNT. **NBR 6023**: Informação e documentação - Referências - Elaboração. Rio de Janeiro, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS - ABNT. **NBR 10520**: Informação e documentação - Citações em documentos - Apresentação. Rio de Janeiro, 2023.

BOCCHINO JR., R. L. **Monads, Categories, and Computation**. [S.l.]: [s.n.], 2025.

HUTTON, G. **Programming in Haskell**. 2. ed. Cambridge: Cambridge University Press, 2016.

JAHN, M. **Monads and their Applications in Haskell**. 2020. Trabalho de Conclusão de Curso (Bachelor's Thesis) – Department of Mathematics, Technische Universität München, Munich, 2020.

MAIA, D. **Applied Category Theory: Monads and Haskell**. 2022.

MALAKHOVSKI, Y. M.; SHALYTO, A. A. Konechnye avtomaty v chistykh funktsionalnykh yazykakh programmirovaniya [Máquinas de estados finitos em linguagens de programação funcional pura]. *RSDN Magazine*, São Petersburgo, n. 1, 2009.

MILEWSKI, B. **Category Theory for Programmers**. [S.l.]: [s.n.], 2020. E-book.

OLIVEIRA, A. G. de. **Haskell: Uma introdução à programação funcional**. São Paulo: Casa do Código, 2017.

PERONE, M.; KARACHALIAS, G. **Crème de la Crem: Composable Representable Executable Machines**. 2023. (arXiv:2307.09090).

WADLER, P. Comprehending Monads. *Mathematical Structures in Computer Science*, v. 2, n. 4, p. 461-493, 1992.

APÊNDICE A — REPRODUÇÃO DOS RESULTADOS

Todos os resultados deste relatório foram obtidos com GHC 9.6.7, cabal 3.12.1.0, `crem` 0.1.0.0, `singletons-base` 3.2 e, para a comparação, o *clang* 14.0.6 que acompanha o GHC, em Windows 11.

Código. Repositório `https://github.com/0ph1s/avionics-crem`, versão `ab02fea` (etiqueta `v0.2.0-pic-parcial`). A versão anterior, `c97981a`, contém a FSM de aviônica, a composição e o simulador; a atual acrescenta as extensões das seções 5.7 (leis de Functor e Applicative), 5.12 (especificação, teste de conformidade e matriz de rejeição), 5.13 (cadeia da bateria, modelo de referência e políticas) e 5.14 (comparação com C e C++). A versão extttab02fea e a etiqueta foram enviadas ao repositório remoto. Porém, uma consulta sem autenticação à página do repositório devolve erro 404, o que indica que ele é privado; os avaliadores não conseguiriam acessá-lo. Também não há arquivamento com identificador permanente. Antes da entrega, convém tornar o repositório público (ou compartilhá-lo com a banca) e arquivar a versão em um serviço que emita identificador permanente.

Comandos, na raiz do pacote:

```
cabal build
cabal run avionics-crem
cabal test avionics-sim-tests
cabal test avionics-spec-tests
cabal test avionics-chain-tests
cabal test avionics-policy-tests
bash scripts/negative_matrix.sh
bash compare-c/run.sh
bash compare-c/run_cpp.sh
cabal build invalid-inflight-shutdown --enable-tests
```

Em ordem: biblioteca e executáveis; seções 1 a 4 e as 17 verificações de leis; simulador (32 verificações); conformidade com a especificação (16); cadeia da bateria ($N = 8$); modelo de referência e políticas ($N = 8$); matriz de 16 pares (8 compilam, 8 rejeitados); mesma máquina em C e em C++ (7 variantes cada); teste de invalidez, que falha de propósito.

O comando `cabal test` sem alvo falha, porque também tenta construir o teste de invalidez, que não compila de propósito; use sempre o alvo explícito.

Arquivos de resultado em `output/`: `negative_matrix.md`, `cadeia_bateria.txt`, `politicas_ambiente.txt`, `comparacao_c.md`, `comparacao_cpp.md` e `comparacao_haskell_tipos.txt`. A especificação do domínio está em `spec/domain_rules.txt`.

Uso de ferramentas de inteligência artificial. Ver a seção DECLARAÇÕES.